

AI-Based Autonomous Exploratory Testing Agent

How each roadmap requirement was fulfilled, the architecture as built, measured results with their caveats, and an honest account of what had to be fixed to make the numbers mean anything.

13 August 2026 Branch `raihan-working` ~14,400 LOC Python · 63 endpoints · 23 graph node types

1 Summary	4 Knowledge graph	7 Challenges
2 Requirement-by-requirement	5 Prompt & token economics	8 Positioning
3 Architecture	6 Test campaigns & results	9 What we need from you

1 · SUMMARY

Where the system stands

All eight roadmap requirements have working implementations, verifiable on a live dashboard. The agent tests any Android application with no documentation at all, and uses an SRS, a UI guide or defect history when they exist. The decisive work this period was not adding capability — it was making the system's own reporting trustworthy.

10/10 work packages implemented	100% autonomy, final campaign	36% on-device pass rate	98 device executions to date	~1¢ model cost per test executed
---	---	-----------------------------------	--	--

The single most important number is **autonomy: 100%**. In the final 25-test campaign, not one execution was lost to a failure of our own agent — every outcome was either the application behaving correctly or the application doing something the specification did not predict. Two campaigns earlier that figure was effectively zero. Until it was high, no defect-discovery number could mean anything, because "the test failed" and "the agent got lost" were indistinguishable.

READ THIS BEFORE QUOTING ANY DEFECT NUMBER

The run produced **16 candidate defects (64%)**. That is *not* a defect-discovery rate — each is an assertion that did not hold and needs human confirmation, and some will prove to be tests whose expectations were wrong. Precision and recall cannot be computed at all until we run against **seeded defects with known ground truth**, which is the first thing we are asking you for (§9).

2 · REQUIREMENT-BY-REQUIREMENT

How each roadmap requirement was fulfilled

For each requirement: what was built, and what evidence exists that it works. All graph writes, endpoints and prompt blocks named below are live in the running system; the screenshots on the following pages show them populated by a real campaign.

ETA-REQ-301 Defect history as a knowledge source

built · awaiting data

- BUILT** Defect nodes with severity, area and root-cause category; POST /ingest/defects, GET /defects/summary, /defects/prone-areas, /defects/context. Defect density weights the retrieval score, and a ## Defect History Context block (2,000-char budget) is injected into generation.
- EVIDENCE** Pipeline runs end to end on synthetic defects. On real data it is **dormant** — the dashboard reports "0 defects · this signal contributes nothing to test generation", which is deliberate honesty rather than a silent zero.

ETA-REQ-302 Navigation tree memory

met

- BUILT** NavTreeNode with shortest-path merge on re-observation and an avoid flag set when a node's success/visit ratio falls below 0.3. /navtree/record-path, /retrieve-path, /failed-paths; two prompt blocks — learned route, and known failed routes.
- EVIDENCE** **123 nodes learned, 95 marked avoid, max depth 42** after a single campaign. Median device steps fell from 50 to 9 across campaigns, which is the behavioural signature this requirement asks for.

ETA-REQ-303 Experiential learning from execution

met

- BUILT** Five persisted stores fed by one executor write: ExecutionLog (full action/screen trajectory, error type, environment), ErrorPattern (mined recurring signatures), StrategyMemory (effectiveness, decay-weighted with a 90-day half-life), CoverageHeatmap (snapshot for trend analysis) and Session (continuity across calls).
- EVIDENCE** 25 execution logs with full routes; three error patterns mined automatically — ASSERTION_FAILURE@Name x12, ASSERTION_FAILURE@Quick Contact x3, CRASH@Contact Editor x1; strategy score 0.64 over 16/25 applications.

ETA-REQ-304 Multi-dimensional knowledge graph

single-app validated

- BUILT** Profile / Platform / Application nodes with TARGETS_* and TESTS_APPLICATION edges; optional dimension parameters on every ingest and retrieval route, defaulting to today's behaviour; MAY_APPLY_TO {transfer_confidence} for cross-dimension transfer; a ## Target Environment prompt block.
- EVIDENCE** Filtering and partitioning verified; **cross-application transfer is not yet validated** because every campaign so far has run against one application.

ETA-REQ-305 Self-healing execution

met

- BUILT** Eight-way failure classification with per-category recovery — alternate navigation route, wait-and-retry, extended timeout, restart-from-checkpoint, or log-as-defect — with the outcome of the recovery attempt written back to the execution log. A retry carries a ## Previous Failure Context block so the second attempt is informed.
- EVIDENCE** Zero agent-attributable failures in the final campaign (§6). Livelock detection was added to this classifier and is the one part still misfiring (§7).

ETA-REQ-306 Regression risk scoring

met

- BUILT** FeatureArea.regression_risk_score computed from defect density, historical failure ratio, recency and navigation instability; exposed at GET /risk/scores and injected as ## Regression Risk Assessment so high-risk areas attract more generation attention.
- EVIDENCE** Two areas scored from live history — contact_management/_validation at 0.28, contact_search at 0.08 — matching where failures actually clustered.

ETA-REQ-307 Test quality & feedback loop

met

BUILT Embedding-cosine semantic dedup (Jaccard retained as a cheap pre-filter) so a reworded duplicate is rejected; per-test defect-discovery rate, execution stability and coverage contribution; strategy scores reinforced when a test exposes a real defect. The eight-way attribution taxonomy (§6) is what makes any of these measurable.

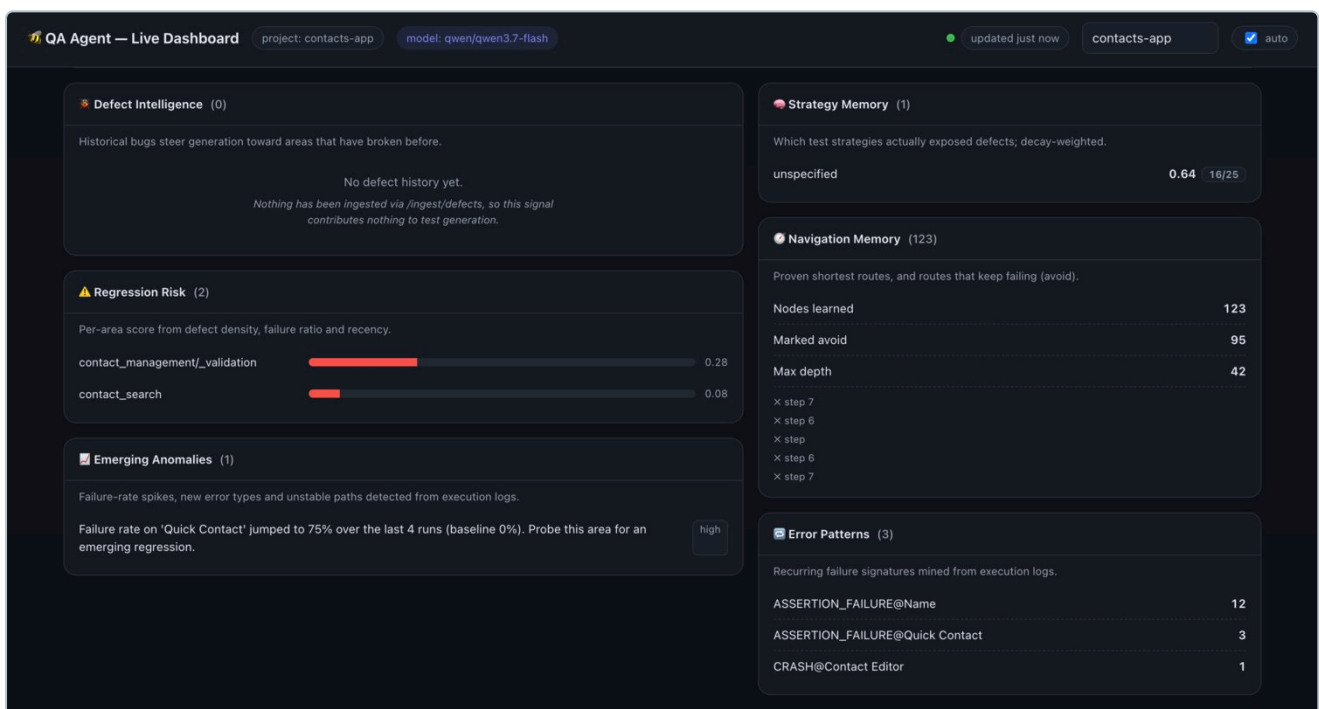
EVIDENCE A dedicated duplicate-check LLM stage runs on every generation (visible in the planner trace). Effectiveness scores are stored per test and exported in the batch CSV.

ETA-REQ-308 Anomaly detection

met

BUILT `AnomalyAlert` nodes from an engine over execution history: failure-rate spikes against a rolling baseline, newly-appearing error types, duration regressions and unstable navigation paths. Alerts are surfaced to generation so the next test investigates them.

EVIDENCE One high-severity alert raised unprompted: *"Failure rate on 'Quick Contact' jumped to 75% over the last 4 runs (baseline 0%). Probe this area for an emerging regression."* Nothing about that area was configured by us.



Requirements 301–308, live. Every learning layer has an operator-visible surface, including when it is empty — defect intelligence states plainly that it is contributing nothing, rather than rendering a zero that looks like a result.

YOUR CORRECTION No knowledge source may be a hard dependency

met

BUILT A pluggable source registry — each source implements `summary_block` / `retrieve` / `prompt_block`, and the planner iterates registered sources instead of a hardcoded list. Correctness signals form a cascade: SRS validation rules → defect-history patterns → UI-guide affordances → generic exploratory heuristics, so there is always a bug oracle even with zero documents. Each source can be switched off from a single settings file.

EVIDENCE Verified with source subsets {SRS+UI, UI-only, none}: each still yields a valid grounded test, and the blocks belonging to removed sources vanish from the prompt. The Figma source is currently disabled by choice — its design guide did not match the shipped application, so live device observation is more accurate.

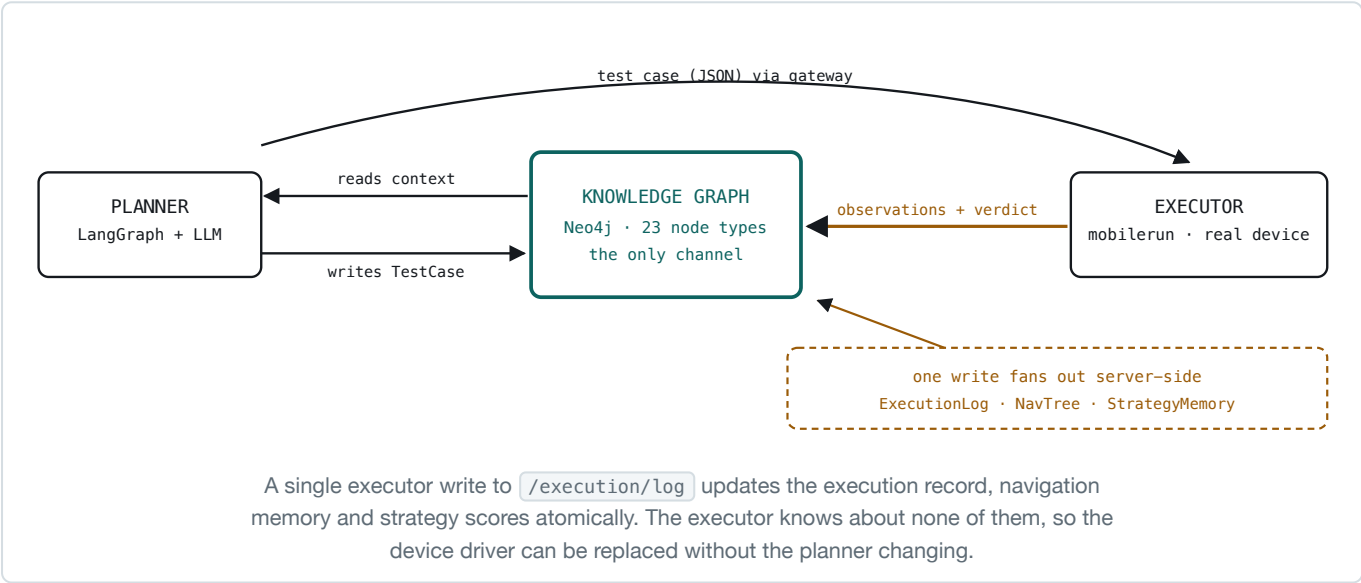
TEAM REQUIREMENT Must work on any generic Android app

BUILT	The Live App Model (§4) builds a navigable UI map purely from on-device accessibility trees and screenshots. Nothing in the planner, prompts or ingestion encodes knowledge of Contacts, of a vendor, or of a document layout — a deliberate constraint we had to enforce twice after catching ourselves keying on the sample SRS's numbering scheme.
EVIDENCE	12 UI states and 44 transitions discovered autonomously with the Figma guide switched off entirely. Target package and project are configuration, not code.

3 · ARCHITECTURE

Four processes, one database

The planner decides what to test; the executor drives the device; neither talks to the other. Everything they share passes through the knowledge graph, and every LLM call is stateless — "learning" means the graph grew, never that a model remembered.



This separation is what makes the roadmap's "every cycle makes it smarter" requirement structural rather than aspirational: there is exactly one place where experience enters the system, so the learning layers cannot drift apart from one another.

QA Agent — Live Dashboard

project: contacts-app

model: qwen/qwen3.7-flash

updated 2s ago

contacts-app

auto

Live Logs (device agent and planner reasoning, side by side — both stream independently)

Device (mobilerun)

300 lines

20:26:37 INFO executor | ROUND 11 RESULT

20:26:37 INFO executor | PLANNER → LOGGING VERDICT & GENERATING NEXT TEST CASE

20:26:53 INFO executor | Planner RAG interaction [round-11]: TC-012 generated

20:26:53 INFO executor | Verdict logged: TC-011 → pass

20:26:53 INFO executor | ROUND 12/25

20:26:53 INFO executor | EXECUTING ON DEVICE: TC-012

20:26:53 INFO executor | Droidrun goal dispatched for TC-012

20:26:58 INFO mobilerun | Trajectory folder: /Users/raihanrashid/Desktop/BUET CSE course all stuff/Capstone/Experiments/logs/trajectories/20260813_202658_13cb8910

20:26:58 INFO mobilerun | Running MobileAgent to achieve goal: Open the Contacts app on this device.

20:27:02 INFO mobilerun | Step 1/50

20:27:12 INFO mobilerun | Step 2/50

20:27:18 INFO mobilerun | Step 3/50

20:27:26 INFO mobilerun | Step 4/50

20:27:44 INFO mobilerun | Step 5/50

Planner

300 lines

14:11:55 → bootstrap_context (round 1)

14:11:55 ✓ bootstrap_context 46ms

14:11:55 → planner_step (round 1)

14:11:59 ✓ LLM openrouter 4.2s ~1980 tok

14:11:59 ✓ planner_step 4.2s

14:11:59 → execute_retrieval (round 1)

14:11:59 ✓ execute_retrieval 92ms

14:11:59 → planner_step (round 2)

14:12:03 ✓ LLM openrouter 3.9s ~1191 tok

14:12:03 ✓ planner_step 3.9s

14:12:03 → generate_testcase (round 2)

14:12:10 ✓ LLM openrouter 6.5s ~5844 tok

14:12:10 ✓ generate_testcase 6.9s

14:12:10 → duplicate_check (round 2)

14:12:16 ✓ LLM openrouter 6.3s ~5041 tok

14:12:16 ✓ duplicate_check 6.5s

Test Cases (25)

ID	TITLE	AREA	VERDICT
TC-025	Validate Save Behavior with Valid Name and Phone but Invalid Email Format (Missing @ Symbol)	contact_management_validation	failed
TC-024	Validate Save Behavior with Valid Name and Email but Invalid Phone Number Format (Non-Numeric Characters)	contact_management_validation	failed
TC-023	Validate Save Behavior with Valid Name, Valid International Phone, and Null Email Field	contact_management_validation	pass
TC-022	Validate Contact Save with Valid Name and Phone but	contact_management_validation	pass

Bugs Found (16)

Validate Save Behavior with Valid Name and Phone but Invalid Email Format (Missing @ Symbol)

contact_management_validation

Validate Save Behavior with Valid Name and Email but Invalid Phone Number Format (Non-Numeric Characters)

contact_management_validation

Validate Name Field Handling of Numeric Strings and Whitespace-Only Input with Valid Phone

contact_management_validation

Validate Contact Save with Valid Name and Email but Missing Country Code in Phone Number

contact_management_validation

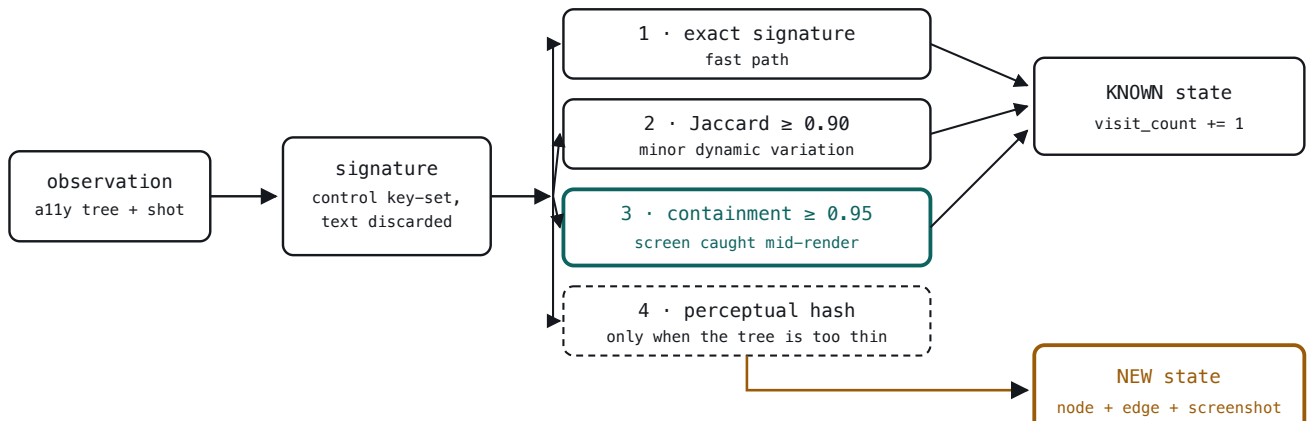
Both halves observable at once. The device agent and the planner stream independently — left, the device stepping through a test; right, the LangGraph nodes with their per-call latency and token counts. Making the planner's reasoning visible is how several of the defects in §7 were found.

4 • KNOWLEDGE GRAPH

How the app model is built, and how a screen is judged unique

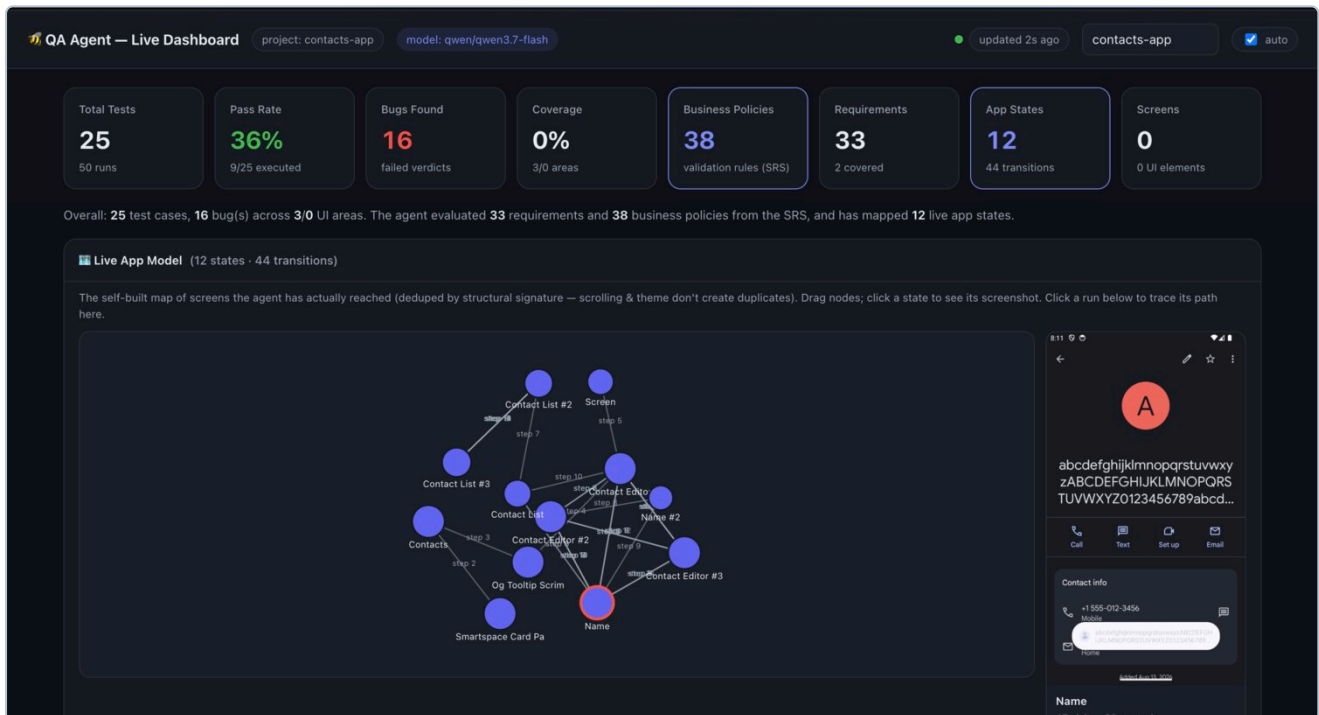
With no design guide, the only reliable knowledge is what the application shows. The agent builds its own map by observing the running device — and the hard part is deciding whether a screen it just saw is one it has already mapped.

Each observation is reduced to a **structural signature**: package, activity, whether a modal surface is open, and the set of controls keyed by (resource_id, class, content-description, clickable). Free text, list contents, row counts and scroll position are discarded. Two properties follow for free — scrolling a contact list past different people is the *same* state, and a light-to-dark theme switch is the *same* state, because the view hierarchy never changed. Identity is structural rather than visual precisely because a vision model would call a dark screen a new screen.



Stage 3 is our addition. **Jaccard** is $|A \cap B|/|A \cup B|$ — symmetric, so it penalises a screen captured before its late-painting chrome appeared and forks a duplicate (0.52 on a real pair).

Containment is $|A \cap B|/\min(|A|,|B|)$, which scores that same subset relationship 1.00 and merges correctly. Applying it above a minimum control count took the mapped-state count on an identical workload from **42 down to 12**.

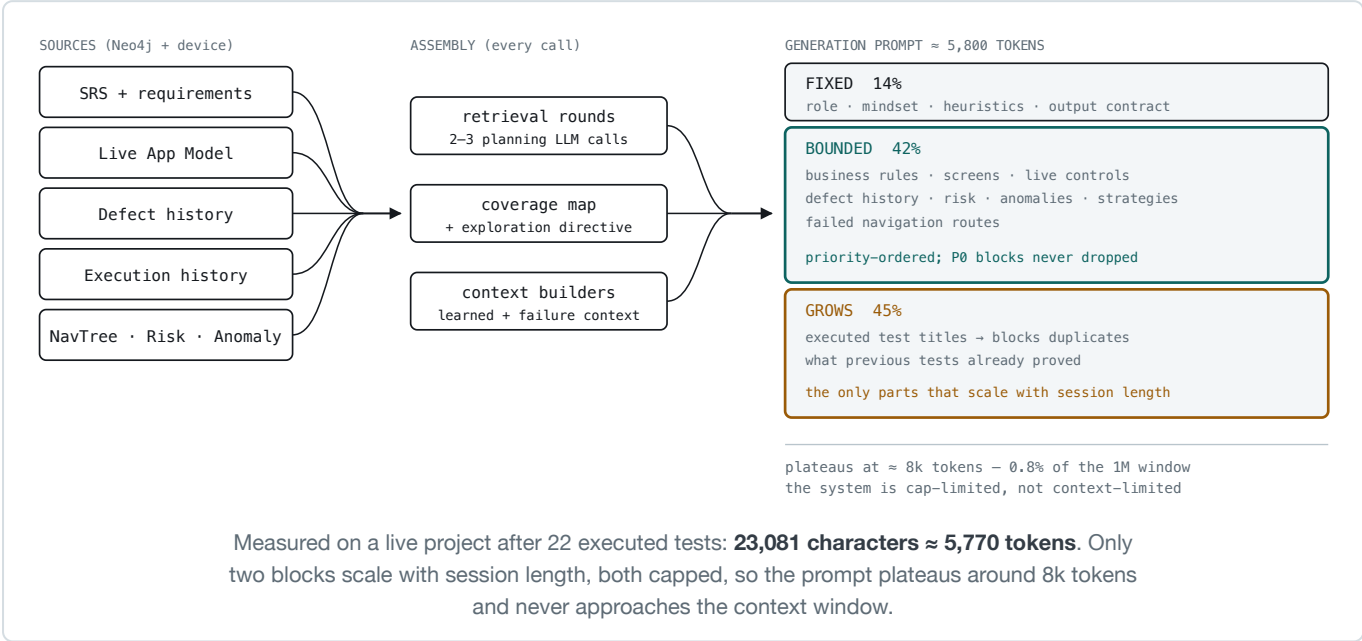


The Live App Model, self-built. 12 states and 44 transitions the agent reached by itself, with no design guide supplied. Each node retains its screenshot and visit count; the labels are derived from content resource-ids rather than window chrome.

The graph holds **23 node types**. From the specification: `Requirement`, `ValidationRule`, `Entity`, `Chunk`, `SRSVersion`. From the device: `UIState` joined by `TRANSITIONS_TO`. From experience: `ExecutionLog`, `NavTreeNode`, `ErrorPattern`, `StrategyMemory`, `AnomalyAlert`, `CoverageHeatmap`, `Session`. Test-to-requirement traceability runs through explicit `COVERS` edges, which is what lets requirement coverage be a first-class metric rather than a proxy such as activity coverage. Current contents: 33 requirements, 38 validation rules, 48 entities, 25 test cases, 12 UI states, 123 navigation nodes.

Spending tokens where they change the outcome

Every generation call rebuilds its prompt from the graph, from scratch. Nothing is remembered between calls, so the prompt *is* the agent's entire working memory — which makes what goes into it, and what gets dropped from it, an architectural decision rather than a detail.



Where the money actually goes — measured, not estimated

Cost is derived from the live billing ledger rather than a model price card — spend in a billing window divided by executions in that same window, so every figure includes retrieval, generation, dedup and device calls:

Running a test is close to free. Reading the specification is not. That is the whole finding, and it is the opposite of what we assumed. The three models in the system differ in price by two orders of magnitude:

MODEL	ROLE	\$/M IN	\$/M OUT	CALLS PER CAMPAIGN
qwen3.7-flash	planner loop	0.030	0.130	108
qwen3-vl-32b-instruct	device agent (vision)	0.104	0.416	~250
qwen3.8-max	SRS extraction	2.000	6.000	~22 (once per document)

Live prices from the provider's model API. The extraction model is 19× the device agent on input and 14× on output — and it is the only one that runs without an output cap.

balance inherited at handover	\$6.500
balance now (verified via billing API)	\$3.815
our spend to date · 88 device executions	\$2.685
device agent · 10.1 steps/test × ~6k tokens	~\$0.0076 (0.8¢)
planner · 108 calls / 384k tokens ÷ 25 tests	~\$0.0006 (0.06¢)
→ cost per test executed	~\$0.008 (~1¢)
→ one 25-test campaign	~\$0.20
most recent billing day, total	\$2.042
less both campaigns (34 executions)	– ~\$0.24
→ one SRS ingestion	~\$1.80

The per-test figure is built from measured quantities, not price-card arithmetic: 10.1 device steps per test (232 stored observations across 23 runs), an accessibility tree averaging 4,686 bytes (~1,170 tokens) per step, a 1080×1920 screenshot, and the planner's 384k tokens logged across 108 calls. It is corroborated independently by the ingestion-free portion of our total spend, which lands at the same figure.

WHY ONE DOCUMENT READ COSTS AS MUCH AS 200 TESTS

The asymmetry is by construction. `extract()` runs a multi-pass extraction **once per sample** — three samples, kept because this artefact underpins every later decision — and each pass splits the specification into ~1,800-character sections and issues **one uncapped call per section**, plus a synthesis and a self-critique call. A final judge call selects the best candidate. On an 8k-character specification that is roughly **22 calls on the most expensive model in the system, with no output ceiling**; the ingest log ends in a 1,800-second timeout, consistent with that volume.

The quality argument for paying it once is sound. What is missing is metering: nothing caps the output length, nothing reduces the sample count once the extraction has stabilised, and nothing prevents an unchanged document from being read again from scratch.

What this means for scaling

Two costs that behave completely differently. **Ingestion is one-time per document** at ~\$1.80 and is paid again only when the specification changes — or, as happened repeatedly during development, when we re-ingest to test a change to the ingestion itself. **Testing is ~1¢ per test** and scales linearly. A nightly regression suite of 500 tests is roughly **\$4**, and 2,000 tests roughly **\$16**, on top of an ingestion that does not repeat.

That also relocates the optimisation target. Within a test execution the device agent is ~92% of the cost and the planner ~8%, so navigation memory cutting median steps from 50 to 9 was a real saving. But against total project spend, **extraction is the dominant line item**, and the levers there are capping its output length, reducing the sample count from three, or caching extraction per document hash so an unchanged specification is never re-read. None of these are implemented; all are straightforward.

Three decisions that keep the cost where it is

- **Tiered models.** Business logic is extracted from the specification *once*, but everything downstream depends on it — so that step uses a large reasoning model with no token cap and best-of-3

sampling, judged for the best result. The per-test planner loop then runs on a model roughly 60× cheaper, and the device agent on a mid-size vision model.

- **One global prompt budget, filled by priority.** A dozen independent caps (120 titles, 25 findings, 8,000 characters of requirements) were replaced by a single token budget filled highest-value-first: the bug oracle and on-screen controls are priority 0 and never dropped, background context truncates only when the ceiling is genuinely reached. The ceiling moved from ~8k to 50k tokens, and every cap that silently discarded data is gone.
- **Vision only where it pays.** The device agent gets screenshots because judging whether an expected result actually occurred is the weakest link. The planner gets none — it reasons about structure, which text conveys better and far more cheaply. At measured volumes vision turned out to be inexpensive: about 0.07¢ per step, so it is not a cost concern.

6 · TEST CAMPAIGNS & RESULTS

Every campaign we have run

Six campaigns, 98 device executions, all preserved on disk as full step-by-step trajectories. Each campaign changed one thing about the agent and was measured against the same application and the same specification.

CAMPAIGN	TESTS	WALL	WHAT CHANGED	OUTCOME
24 Jul · 22:25	6	67m	first end-to-end wiring	pre-instrumentation smoke run
25 Jul · 01:42	4	14m	graph write-back	pre-instrumentation smoke run
12 Aug · 22:42	25	107m	first full campaign with attribution	0% pass — 82% of device steps unparseable
13 Aug · 03:03	29	84m	non-reasoning vision model; livelock detector	9% → 20% pass , autonomy 91% → 96%
13 Aug · 19:02	9	16m	—	cancelled by us after spotting bad preconditions and unreadable logs
13 Aug · 19:55	25	61m	precondition filter, containment matching, device reset, log reassembly	36% pass · 100% autonomy · 0 silent fallbacks

Reconstructed from on-disk trajectory directories, one per device execution. The fifth campaign was aborted deliberately — inspecting a run and stopping it is cheaper than letting a known-bad configuration produce 25 misleading results.

Why campaigns stop at 25 tests

25 is a deliberate ceiling, not a technical limit — the agent will run indefinitely. Three reasons hold it there for now.

The budget pressure is re-ingestion, not test volume. A 25-test campaign costs about \$0.20, so length is not what constrained us — we have spent \$2.69 of an inherited \$6.50 prepaid balance, and the great majority of that went on re-reading the specification each time we changed how ingestion works (~\$1.80 a time), not on running tests. Nearly every improvement in the table above came from *changing* the agent between campaigns rather than from running a longer one, and each of those changes to the ingestion path cost far more than the campaign that followed it.

25 is what one engineer can review by hand. Until we have seeded defects, the only way to know whether a "candidate defect" is real — and whether the test was checking the right requirement in the first place — is to read every test, its route and its verdict. That is a couple of hours for 25 tests and infeasible for 200. Every campaign therefore exports a review sheet with blank rubric columns for exactly this purpose.

The prompt's dedup memory saturates near this scale. Executed-test titles are capped at 120 and prior findings at 25; past those, older entries drop out and the planner can regenerate a test it has already tried. Raising the caps is nearly free at current sizes, but we would rather raise them deliberately once we can measure the effect than discover the regression in a 200-test run.

The final campaign

9	16	0	0	122s	12
passed	candidate defects, unvalidated	agent failures	silent fallbacks	mean per test	UI states mapped

QA Agent — Live Dashboard

project: contacts-app

model: qwen/qwen3.7-flash

updated 2s ago

contacts-app

auto

Execution Paths (25 runs)

Each test's real route through the app. Click one to highlight its path on the graph above — a quick way to spot redundant or wrong states.

failed

TC-025 — Validate Save Behavior with Valid Name and Phone but Invalid Email Format (Missing @ Symbol)

7 states · 11 steps · 103s · ASSERTION_FAILURE

Smartspace Card Pager → Contacts → Og Tooltip Scrim → Contact Editor → Contact Editor #3 → Contact Editor #2 → Name

show device steps

failed

TC-024 — Validate Save Behavior with Valid Name and Email but Invalid Phone Number Format (Non-Numeric Characters)

6 states · 14 steps · 155s · ASSERTION_FAILURE

Smartspace Card Pager → Contacts → Og Tooltip Scrim → Contact Editor → Contact Editor #2 → Name

show device steps

pass

TC-023 — Validate Save Behavior with Valid Name, Valid International Phone, and Null Email Field

7 states · 10 steps · 98s

Smartspace Card Pager → Contacts → Og Tooltip Scrim → Contact Editor → Contact Editor #3 → Contact Editor #2 → Name

show device steps

pass

TC-022 — Validate Contact Save with Valid Name and Phone but Null/Empty Email Field (Distinct from Empty String)

7 states · 10 steps · 103s

Planner Execution Trace (each generation run: LangGraph nodes, LLM cost, retrievals — newest first)

ok

/agent/next-testcase

20:55:57

15.9s total · 4 LLM (15.0s) · 13 RAG · ~17,911 tok

ok

/agent/next-testcase

20:52:58

15.7s total · 4 LLM (15.4s) · 13 RAG · ~17,834 tok

ok

/agent/next-testcase

20:50:56

16.6s total · 4 LLM (16.0s) · 13 RAG · ~17,616 tok

ok

/agent/next-testcase

20:48:51

16.5s total · 4 LLM (15.8s) · 14 RAG · ~18,724 tok

ok

/agent/next-testcase

20:46:41

35.2s total · 5 LLM (34.3s) · 16 RAG · ~22,092 tok

Every run's real route is retained. Route, state count, step count, duration and error type per test — this is what makes a failure reviewable rather than merely counted. The planner trace beneath it accounts for every LLM call and its token cost.

How outcomes are attributed

A failed run is not a discovered defect. Every execution falls into one of eight categories, and only two are evidence about the application:

OUTCOME CATEGORY	ATTRIBUTED TO	CANDIDATE DEFECT?
ASSERTION_FAILURE, CRASH	The application	yes
NAVIGATION_FAILURE, ELEMENT_NOT_FOUND, TIMEOUT, NAVIGATION_LIVELOCK	Our agent	no — lowers autonomy

OUTCOME CATEGORY	ATTRIBUTED TO	CANDIDATE DEFECT?
PRECONDITION_NOT_MET, PERMISSION_DENIED, STEP_LIMIT_EXCEEDED	The environment	no – excluded

This taxonomy is why the numbers above can be read at all. It resolves the question that made the project hard to measure honestly: a task the agent could not complete may be a genuine defect, a limitation of the agent, or an impossible precondition — and those three must never be summed.

Execution Paths (25 runs)

Each test's real route through the app. Click one to highlight its path on the graph above — a quick way to spot redundant or wrong states.

CLICKED ON TEXT: SKIP | CLASS: BUTTON | TYPE: UNKNOWN | COORDINATES: (500, 1770)

The Contacts app is open, but it's showing a setup screen asking to sign in or skip. Since the goal is to test contact creation and we don't need to sync with Google for this test, I'll skip the setup to proceed to the main contacts list.

3 click index 17 ok

Clicked on Text: 'Create contact' | Class: ImageButton | Type: unknown | Coordinates: (943, 1531)

I'm now on the main Contacts screen, which shows "No contacts yet" and has a floating action button (+) to create a new contact. This matches the precondition of being on the main contacts list screen.

Next, I need to navigate to the 'Contact Editor' screen by tapping the '+' button to add a new contact.

4 type index 11 · "Jane" ok

Text typed successfully (clear=False)

I'm now on the 'Contact Editor' screen, which matches the requirement. The next step is to enter a valid name in the Name field.

I'll start by entering "Jane Doe" — first name "Jane" and last name "Doe" — into the respective fields.

5 type index 12 · "Doe" ok

Text typed successfully (clear=False)

Step-level reasoning is recorded, not just the verdict. Each device action carries the agent's justification, so a wrong outcome can be traced to the step where its understanding diverged. Recovering this was itself a fix — the device agent streams one token per log record, which arrived shredded until we reassembled the stream.

Each campaign also exports a machine-readable CSV — test id, title, verdict, error type, attribution, device steps, distinct states, duration, cited requirement ids and the full screen route — plus a review sheet with blank rubric columns. Before this, run data lived only in the graph and was destroyed by the next clean-slate wipe.

7 · CHALLENGES

What had to be fixed

Most of this period's engineering went into defects in our own system that were invisible precisely because the system handled them gracefully. Every one was found by inspecting real output, not by reading code.

DEFECT	HOW IT STAYED HIDDEN	CONSEQUENCE
86% of the SRS discarded at ingestion	Chunking kept only lines matching a requirement-ID pattern and dropped every prose paragraph; the endpoint reported success.	Retrieval served headings without the requirements under them
A deleted document kept serving requirements	Ingestion appended rather than replaced, keyed by filename. 17 of 19 chunks came from a file removed weeks earlier.	The model cited requirements that no longer existed
State identity overwritten on merge	A near-match adopted the newcomer's signature, so a later identical observation no longer matched and forked.	Self-inflicted state explosion — 42 states where 12 were real
Requirement coverage read 0% for days	The one call that creates <code>COVERS</code> edges sat inside a bare <code>except: pass</code> .	Traceability, our main differentiator, appeared not to work

DEFECT	HOW IT STAYED HIDDEN	CONSEQUENCE
The device reset reset nothing	It cleared the app package, but contact data lives in a separate content provider. The command returned "Success".	33 contacts accumulated; clean-state tests unsatisfiable
Reasoning tokens starved the answer	A 2,048-token output cap was consumed by a reasoning model's private scratchpad, truncating the JSON mid-object.	A stronger model produced <i>fewer</i> requirements than a weaker one

THE PRINCIPLE WE ADOPTED: DEGRADE, BUT NEVER QUIETLY

Removing the error handling would be the wrong lesson — one malformed screen must not end a two-hour unattended run. Instead every fallback now records what capability was lost and at what severity, surfaced on the dashboard and in the end-of-run report. A run that "succeeded" while operating on worse data is visibly marked untrustworthy. The final campaign reported **zero silent fallbacks** — a statement that is only meaningful because the mechanism to detect them exists.

Three lessons worth carrying forward

A model that reasons in-band breaks a strict output parser. The device model's `<think>` output corrupted tool-call parsing, so 82% of steps did nothing — the agent narrated intentions and never acted. This presents as "the agent is bad at the task", not as a parsing error, which is why it survived a whole campaign. A non-reasoning vision model took parse failures to zero and the median step count from 50 to 9.

Fixing a false positive invites a false negative. Livelock detection originally aborted any run that revisited a few screens, which killed the legitimate test *"add ten contacts and observe the list"* — structurally two screens alternating. Adding a content-progress signal fixed that and introduced the opposite error: an agent retyping the same field forever looks like progress because the text keeps changing. One run cycled 43 observations across 5 states for the full budget and was recorded as a crash. Still open.

Generality has to be enforced, not assumed. An early fix to the chunker still keyed on the sample SRS's numbering conventions — a system that works on documents shaped like ours, not a generic one. It was rewritten to know nothing about requirement conventions and to retain 100% of source text regardless of format.

Relationship to published work

What we adopt. Our structural signature is a re-derivation of *STOAT* (ESEC/FSE 2017), which treats only structurally different pages as distinct states. Threshold-based similarity over a state abstraction is standard — *Judge* (ACM TOSEM) treats abstraction as a first-class problem, and Jaccard with a tuned threshold appears in Android GUI testing at 0.75–0.82 against our 0.90. *APE* (ICSE 2019) refines its abstraction at runtime rather than fixing it in advance, which is where our cascade is heading. Attribution before measurement follows recent LLM GUI-agent evaluation work.

Where we differ. Three things, stated conservatively:

- **Requirement-traceable exploratory testing.** The literature measures activity or code coverage; none of it can state which *requirement* a generated test exercises. Our `COVERS` edges plus SRS

drift detection make requirement coverage a first-class metric for an exploratory agent — an intersection we have not found occupied.

- **Multi-source graceful degradation as an architecture.** Related work assumes a fixed input, usually the APK alone. Our source registry runs identically with a specification, a UI guide, defect history, any subset, or none.
- **Containment matching for partial observations.** A standard set measure, but we did not find it used this way in GUI testing. We present it as our adaptation, not as established practice.

A CHECK WE RAN ON OURSELVES

We re-validated the sources this design leans on. *STOAT*, *APE*, *GPTDroid* (ICSE 2024) and *Judge* are peer-reviewed at top-tier venues. Several recent GUI-agent papers we drew framing from are **unrefereed arXiv preprints** with very small benchmarks; we treat their methods as informative and their numbers as not yet a baseline, and do not cite them as evidence for our own claims.

9 • WHAT WE NEED FROM YOU

Two requests, and what they unblock

Both remaining gaps in the roadmap are blocked on inputs we cannot produce ourselves. Neither is large, and each converts a capability we have already built from "implemented" into "demonstrated".

1 — A defect or issue-tracker export

ETA-REQ-301 is fully built and completely dormant: no defect data has ever been ingested, so defect-density-weighted exploration has never actually influenced a test. An export from any application we can also run against — id, title, area or component, severity, status, and ideally the resolution or root cause — would activate it immediately. Format is flexible; the ingestion endpoint already normalises. Even a few hundred historical issues would let us show tests skewing toward defect-prone areas versus a no-defect baseline, which is the requirement's stated acceptance criterion.

2 — A build with seeded defects, for precision and recall

This is the more important of the two. Every quality number in this report is bounded by the fact that **we have no ground truth**: we can say a test failed, and we can say the agent did not cause the failure, but we cannot say whether a real defect exists, and we cannot detect a defect the agent *missed*. Recall is currently unmeasurable in principle.

What would fix it is a build of a target application with **8–12 known defects deliberately introduced** and a list of what they are and where — a relaxed validation rule, a broken duplicate check, a dropped required field, an incorrect boundary. With that we run three arms against a fixed budget: the full agent, the agent with its memory layers disabled, and a random-exploration baseline. That produces **precision, recall and F1 against ground truth**, and the ablation gives direct evidence for the roadmap's central claim that every execution cycle makes the agent smarter. The measurement infrastructure is already built and the campaigns are cheap — under \$1 of model spend for all three arms, since the specification only needs ingesting once. We can also seed the defects ourselves if you can point us at a suitable application and confirm that is acceptable.

3 — Scope guidance on the specification

Smaller, but it distorts a headline metric. The sample SRS contains activation and SIM-region requirements that may not be reachable through the application interface at all. While they remain in the denominator they depress requirement coverage permanently regardless of how well the agent performs. A note on which requirement classes are considered in scope for UI testing would let us report that figure honestly.

Known open issues on our side

ISSUE	CONSEQUENCE	PRIORITY
Requirement coverage reads 2 of 33	Traceability works mechanically, but few generated tests cite requirement ids, so our main differentiator is under-demonstrated	high
Livelock false negative	An agent repeating one action indefinitely is recorded as a crash, inflating the apparent defect count	high
Planner invents cross-screen premises	Tests occasionally target controls that do not exist on the screen named in the step, wasting executions	medium
Impoverished observations	Recorded UI events omit clickable, content-description and package, weakening state identity and screen labelling	medium
Extraction is non-deterministic	The same document yields different requirement counts between runs, breaking reproducibility of any experiment built on it	medium
Cross-application transfer unvalidated	ETA-REQ-304 is built but every campaign has run against one application	medium

Primary references

1. Su et al., *Guided, Stochastic Model-Based GUI Testing of Android Apps* (STOAT), ESEC/FSE 2017.

2. Gu et al., *Practical GUI Testing of Android Applications via Model Abstraction and Refinement* (APE), ICSE 2019.

3. Judge: *Effective State Abstraction for Guiding Automated Web GUI Testing*, ACM TOSEM.

4. Liu et al., *Make LLM a Testing Expert* (GPTDroid), ICSE 2024.

5. *Beyond Static GUI Agent: Evolving LLM-based GUI Testing via Dynamic Memory*, ASE 2025.

Account balance (\$3.815 remaining) and model prices read from the OpenRouter API, 13 August 2026. The per-test figure is built bottom-up from live prices and measured payloads — step counts and accessibility-tree sizes from `logs/trajectories/`, planner token totals from the gateway log — and is accurate to roughly $\pm 0.3\text{c}$; the ingestion figure is obtained by subtracting those from a billing day and carries wider uncertainty, perhaps $\pm \$0.30$. Campaign history reconstructed from `logs/trajectories/`. Final results exported to `logs/batch_contacts-app_20260813_205801.csv`. Prompt composition measured with `scripts/dump_prompt.py`.